

TrafficClaw: A Generalizable LLM Agent in the Unified Physical Environment for Urban Traffic Control

Siqi Lai¹ Pan Zhang¹ Yuping Zhou¹ Jindong Han² Yansong Ning¹ Hao Liu¹
slai125@connect.hkust-gz.edu.cn

¹The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, Guangdong, China

²Shandong University, Jinan, Shandong, China

Abstract

Large language model (LLM) agents have shown strong capabilities in long-horizon reasoning, tool use, and decision-making in digital environments, yet extending them to physically grounded systems remains challenging. Unlike web, code, or game environments, where objectives are often weakly coupled, physical systems evolve through tightly coupled dynamics in which local interventions propagate across interacting subsystems over time. Urban traffic control exemplifies this challenge, as traffic signals, freeways, public transit, and taxi systems continuously interact through shared spatial infrastructure and temporal mobility demand. Existing optimization, reinforcement learning (RL), and LLM-based approaches are largely designed for isolated subsystems, limiting coordinated reasoning and system-level optimization. We propose *TrafficClaw*, a LLM-based generalizable traffic control agent for physical urban systems. TrafficClaw operates within a unified traffic environment that exposes coupled urban dynamics and feedback, performs executable spatiotemporal reasoning with persistent memory for long-horizon adaptation, and leverages multi-stage agentic RL for coordinated system-level optimization. Experiments across three metropolitan regions and six traffic-control tasks demonstrate strong generalization, robustness, and cross-subsystem coordination. Our project is available at <https://github.com/usail-hkust/TrafficClaw>.

1 Introduction

Large language model (LLM)-based systems have recently emerged as a promising paradigm for general-purpose agents, demonstrating strong capabilities in long-horizon planning, tool use, and reasoning (Hu et al., 2025; Xi et al., 2025; Fang et al., 2025; Pang et al., 2025). However, most existing LLM agents are developed in separate digital environments such as search engines (Li et al., 2025b),

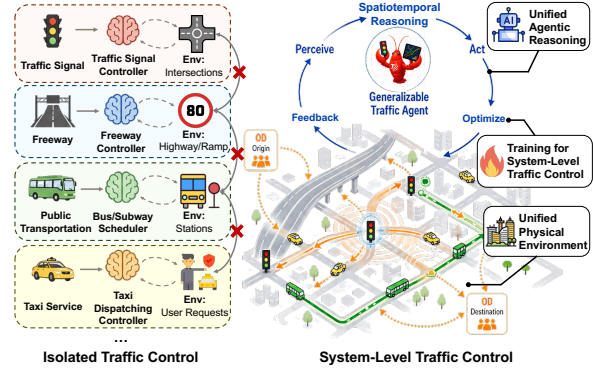


Figure 1: Isolated vs. unified traffic control.

games (Dong et al., 2024), and computer-use systems (Steinberger and Contributors, 2025), where tasks remain modular, actions are mediated through predefined interfaces, and feedback is largely localized to the current environment state. Such environments provide limited support for modeling physical-world dynamics, where local interventions can propagate across interacting subsystems over time. Extending LLM agents from digital task execution to physical-world decision-making therefore requires environments that explicitly expose coupled state transitions, temporal feedback, and cross-subsystem interactions.

Urban traffic exemplifies this challenge in physical environments. Heterogeneous subsystems (e.g., traffic signals, freeways, public transit, taxi services) interact under shared spatial infrastructure and temporal mobility demand constraints. Existing traffic control methods, including optimization-based (Koonce et al., 2008; Papageorgiou et al., 1991), RL-based (Farazi et al., 2021; Wang and Sun, 2022; Jin et al., 2019; Li et al., 2017; Belletti et al., 2017), and recent LLM agents (Lai et al., 2025; Yuan et al., 2025; Han et al., 2025; Feng et al., 2025), are largely developed for isolated subsystems. As shown in Figure 1, such isolated formulations fail to capture coupled physical dynamics across subsystems, limiting coordinated

reasoning and preventing system-level decision-making. These limitations motivate a unified agentic framework that embeds traffic control within a shared physical environment for cross-subsystem reasoning and system-level optimization.

Bringing LLM agents into urban traffic control raises three challenges. First, isolated subsystem environments cannot capture coupled urban mobility dynamics. Generalizable traffic control therefore requires a unified physical environment where heterogeneous subsystems interact through shared spatial infrastructure and temporal mobility demand constraints. Second, subsystems exhibit distinct spatial structures and temporal dynamics. Agents need to reason coherently across heterogeneous subsystems. Third, unified optimization remains difficult because different subsystems involve distinct objectives, reward scales, and potentially conflicting goals, where local improvements may degrade global performance. Effective traffic control therefore requires coordinated system-level optimization across heterogeneous subsystems.

To address these challenges, we propose *TrafficClaw*, a LLM-based generalizable traffic control agent for physical urban systems. TrafficClaw operates within a unified physical traffic environment that integrates heterogeneous subsystems through shared spatial infrastructure and temporal mobility demand dynamics. The environment models coupled urban traffic dynamics, allowing local interventions to propagate across subsystems and induce network-wide feedback. This environment provides a shared interaction space for long-horizon reasoning, cross-subsystem coordination, and system-level optimization.

Within this unified environment, we develop a unified LLM agent for traffic control. Instead of using predefined pipelines for each subsystem, the agent performs unified analysis across spatial, temporal, and cross-subsystem dimensions through flexible code-based analytical modeling. This process generates structured diagnostics of how congestion, demand, and interventions propagate throughout the network, enabling coherent reasoning across heterogeneous subsystems. Meanwhile, the spatiotemporal memory accumulates reusable procedural knowledge, including congestion patterns, coordination strategies, and failure modes, across episodes. By integrating spatiotemporal reasoning with accumulated memory, TrafficClaw continuously self-improves its decision-making and adapts to diverse traffic dynamics.

To effectively translate analytical reasoning into actionable policies, we adopt a multi-stage training strategy. The agent is initialized using diverse multi-task trajectories spanning spatial regions, temporal patterns, and task configurations, bootstrapping generalizable reasoning across traffic subsystems. It is then optimized via agentic RL under system-level objectives that jointly model traffic efficiency and coordination quality. This interaction-driven training enables unified optimization across heterogeneous and potentially conflicting objectives, yielding system-aware control policies.

Our contributions are threefold. (1) We introduce an LLM agent that operates within a unified urban traffic environment, integrating heterogeneous subsystems into a shared physical system. To our knowledge, this is the first framework enabling generalizable traffic control with LLM agents in a unified environment. (2) We propose a traffic control agent with executable spatiotemporal reasoning, persistent memory mechanism, and system-level agentic RL, enabling unified diagnostics, long-horizon adaptation, and coordinated decision-making across coupled and heterogeneous subsystems. (3) Through extensive experiments on three metropolitan regions and six traffic-control tasks, we demonstrate that TrafficClaw achieves strong generalization and robustness across traffic subsystems and task configurations.

2 Preliminary

Definition 1. Agentic Traffic Control. We formulate traffic control as a partially observable Markov decision process (POMDP) defined by a tuple $\langle S, O, A, \mathcal{T}, \mathcal{F}, R \rangle$:

- **State:** S is the traffic system state space, and $s_t \in S$ denotes the traffic state at time t capturing spatiotemporal conditions of the controlled traffic infrastructure;
- **Observation:** O is the observation space, and $o_t \in O$ denotes the observation at time t obtained from sensors or monitoring systems, where $o_t \subseteq s_t$ is a subset of the state;
- **Action:** A is the control action space, and $a_t \in A$ denotes the control action applied at time t (e.g., signal phase change);
- **Task:** $\tau \in \mathcal{T}$ is a traffic control task, specifying an operational goal such as flow optimization in traffic signal control;
- **Transition function:** $\mathcal{F} : S \times A \rightarrow S$ models the traffic dynamics, defining how the state

evolves in response to control actions, capturing physical traffic flow dynamics and congestion propagation under the applied control;

- **Reward function:** $R(s_t, a_t) = f_{\text{reward}}(s_t, a_t)$ evaluates the immediate performance of executing action a_t under state s_t .

Given a traffic control POMDP $\langle S, O, A, \mathcal{F}, R \rangle$ and a user query specifying a task $\tau \in \mathcal{T}$, agentic traffic control aims to perform automatic reasoning and decision-making to meet the user's request. The agent observes historical traffic observations $\mathcal{O}_{1:t} = \{o_0, \dots, o_t\}$ and past actions $\mathcal{A}_{1:t} = \{a_0, \dots, a_{t-1}\}$. At each decision step t , the agent produces a sequence of control actions:

$$\mathcal{A}_{t:t+H} = \pi(\tau, \mathcal{O}_{1:t}, \mathcal{A}_{1:t}), \quad (1)$$

where $\pi(\cdot)$ is the policy function that maps the input to the control action sequence. The goal is to optimize task-specific performance under physical and operational constraints.

3 Unified Traffic Environment

3.1 Urban Traffic System Modeling

Urban traffic consists of heterogeneous subsystems coupled through spatial infrastructure and mobility demand constraints. We construct a unified traffic environment that integrates subsystems into a shared interactive framework for closed-loop perception, analysis, action, and optimization.

Definition 2. Traffic Subsystem. A traffic subsystem refers to a basic component of urban transportation that can be controlled to optimize traffic efficiency. Formally, subsystem n is defined by its action space $A^{(n)}$ and action $a_t^{(n)} \in A^{(n)}$ at time t .

Definition 3. Unified Traffic Environment. A unified traffic environment integrates N heterogeneous traffic subsystems into a single shared dynamical system operating over a common physical traffic network. It serves as the operational platform for the agentic traffic control process. Formally, it is defined by a tuple $\langle N, S^{\text{env}}, S^I, S^M, \{A^{(n)}, \Pi_I^{(n)}, \Pi_M^{(n)}\}_{n=1}^N, \mathcal{F} \rangle$:

- **Subsystems:** N is traffic subsystem number;
- **Environment state:** S^{env} is the global traffic state space over the entire traffic network;
- **Infrastructure:** I denotes the common spatial infrastructure, including roads, lanes, stations, and zones. S^I is its state space, with infrastructure state $s_t^I \in S^I$. $\Pi_I^{(n)}$ maps subsystem n to the infrastructure it observes or controls;

- **Mobility:** M denotes the time-indexed mobility domain. S^M is its state space, with mobility state $s_t^M \in S^M$. $\Pi_M^{(n)}$ maps subsystem n to the mobility it serves or reshapes;
- **Transition functions:** $\mathcal{F}^{\text{env}}$ is the unified transition function induced by local subsystem transitions over the shared infrastructure and demand states, formally defined as:

$$s_{t+1}^{I,(n)} = \mathcal{F}_I^{(n)}(s_t^{\text{env}}, \Pi_I^{(n)} s_t^I, a_t^{(n)}), \quad (2)$$

$$s_{t+1}^{M,(n)} = \mathcal{F}_M^{(n)}(s_t^{\text{env}}, \Pi_M^{(n)} s_t^M, a_t^{(n)}), \quad (3)$$

$$s_{t+1}^{\text{env}} = \mathcal{F}^{\text{env}}(s_t^{\text{env}}, s_t^I, s_t^M, \{a_t^{(n)}\}_{n=1}^N), \quad (4)$$

where local transitions $\mathcal{F}_I^{(n)}$ and $\mathcal{F}_M^{(n)}$ describe how subsystem actions alter their associated infrastructure and mobility dynamics. If two subsystems operate on the same road, station, zone, or mobility demand, their local interventions induce a shared infrastructure-mobility state, which together reshapes the global environment.

Definition 4. Urban Traffic Control. Urban traffic control is defined as a POMDP tuple $\langle S^{\text{env}}, S^I, S^M, O, \{A^{(n)}\}_{n=1}^N, \mathcal{T}', \mathcal{F}^{\text{env}}, R \rangle$, where $\mathcal{T}' = \{\tau_k\}_{k=1}^K$ specifies K ($K \geq 1$) control tasks from different subsystems. The goal is to optimize the local subsystem- and system-level performance under coupled network-wide dynamics through a runtime loop that continuously maps observation, analysis, action, and feedback into improved urban traffic efficiency.

3.2 Multi-Mode Mobility Demand Modeling

Mobility demand links traveler behavior to subsystem dynamics, which specifies how travelers enter and interact with the shared physical traffic system across space and time (Xu et al., 2025, 2026). We generate urban origin–destination (OD) demand from official government household travel surveys (e.g., NYC Mobility Survey (New York City Department of Transportation, 2022)), using empirical priors on trip volumes and mode preferences to ground demand in real-world behavior.

We model aggregate inter-region demand using a gravity-based formulation (Anderson, 2011; Zhao et al., 2024; Piao et al., 2025; Bougie and Watanabe, 2025). Let i and j denote origin and destination regions. The demand between them is modeled as the product of the activity levels of the two regions and is impeded by travel distance:

$$D_{ij} \propto \frac{Q_i \cdot Q_j}{e_{ij}}, \quad (5)$$

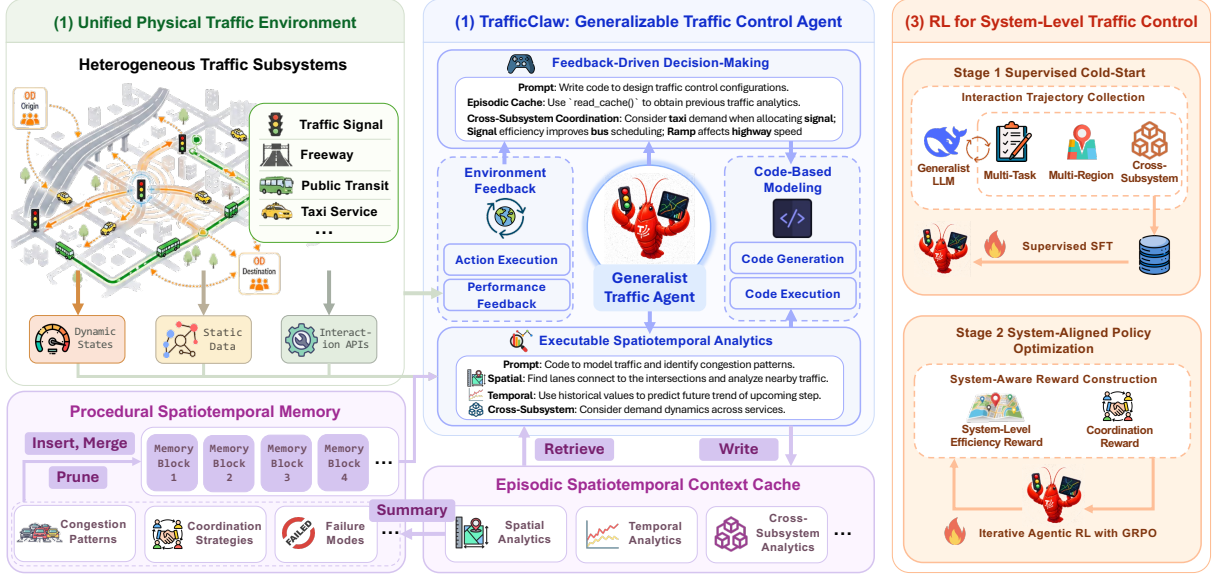


Figure 2: The framework overview of TrafficClaw.

where Q_i and Q_j denote regional activity intensities computed from population density and POI concentration, and e_{ij} denotes the travel impedance (*i.e.*, free-flow travel time).

We assign the travel mode (*e.g.*, walk, vehicle, subway) of each trip from mobility survey statistics and construct the mode-specific OD demand as:

$$D_{ij}^{(m)} = D_{ij} \cdot p(m | c_{ij}), \quad (6)$$

where $m \in \mathcal{M}$ denotes the travel mode and $p(m | c_{ij})$ denotes the probability of selecting mode m conditioned on the trip purpose–distance category c_{ij} . These mode-specific demands initialize the mobility state s_t^M , which is subsequently updated through coupled subsystem interactions. Consequently, mobility demand dynamically interferes with evolving traffic dynamics.

4 Generalizable Traffic Control Agent

Figure 2 presents an overview of TrafficClaw for generalizable traffic control. (1) The agent performs executable spatiotemporal reasoning to generate structured diagnostics and feedback-grounded actions through closed-loop agent-environment interactions. (2) A spatiotemporal memory mechanism maintains cross-episode analytical context, accumulating procedural knowledge for self-improvement in long-horizon control. (3) A multi-stage training strategy starts with supervised initialization and is followed by agentic RL under system-level objectives, enabling unified optimization across heterogeneous subsystems and aligning local decisions with system-level traffic efficiency.

4.1 Agentic Spatiotemporal Reasoning

4.1.1 Executable Spatiotemporal Analytics

To enable system-level traffic control, the agent needs to reason about (1) where inefficiencies emerge, (2) how they evolve over time, and (3) how local interventions affect system-level efficiency. Since predefined pipelines cannot adequately model heterogeneous dynamics and unseen scenarios, we formulate decision-making as an executable reasoning loop that generates and executes code for analytical modeling. This process produces structured diagnostics of coupled traffic dynamics through a unified analytical interface, enabling coherent cross-subsystem reasoning and coordinated decision-making.

Given $\mathcal{O}_{1:t}$ and $\mathcal{T}'$, the agent first generates and executes analytical code over the observed spatiotemporal traffic data. Concretely, the agent is instructed to generate executable code for analytical modeling, including: (1) *spatial modeling* $\Phi_{\text{space}}(\cdot)$ captures network topology and spatial dependencies; (2) *temporal modeling* $\Phi_{\text{time}}(\cdot)$ extracts dynamic patterns from traffic evolution; and (3) *cross-subsystem modeling* $\Phi_{\text{cross}}(\cdot)$ characterizes how local interventions propagate across subsystems. Rather than relying on a fixed pipeline, the agent implements the required analytics based on the task objectives and observed dynamics:

$$Y_t^{\text{ana}} = \Phi_{\text{ana}}(\mathcal{O}_{1:t}, \mathcal{T}'), \quad (7)$$

where $\Phi_{\text{ana}}(\cdot)$ denotes a composition of analytical modeling processes (*e.g.*, $\Phi_{\text{ana}} = \Phi_{\text{space}} \circ \Phi_{\text{time}} \circ \Phi_{\text{cross}}$), and Y_t^{ana} is a structured diagnostic that

summarizes the spatiotemporal state of the system and serves as the basis for downstream decision-making. The detailed prompt and analytical modeling processes are in Appendix L.

4.1.2 Feedback-Driven Decision-Making

Based on the analytical outputs, the agent generates control actions grounded in inferred spatiotemporal traffic dynamics. Formally, given Y_t^{ana} and task set $\mathcal{T}'$, the actions at the next step are produced as:

$$\mathcal{A}_t = \Phi_{\text{act}}(Y_t^{\text{ana}}, \mathcal{O}_{1:t}, \mathcal{T}'), \quad (8)$$

where $\Phi_{\text{act}}(\cdot)$ denotes the executable code of the control policy generated by the agent. TrafficClaw evaluates them through a feedback-driven rollout process within the unified environment:

$$\hat{s}_t^{\text{env}} = \mathcal{F}^{\text{env}}(s_{t-1}^{\text{env}}, \mathcal{A}_t), \quad (9)$$

$$r_t = \sum_{\tau_k \in \mathcal{T}'} R_{\tau_k}(\hat{s}_t^{\text{env}}, \mathcal{A}_t) + R_{\text{env}}(\hat{s}_t^{\text{env}}), \quad (10)$$

where $\hat{s}_t^{\text{env}}$ denotes the rolled-out traffic state, and $R_{\tau_k}(\cdot)$ and $R_{\text{env}}(\cdot)$ evaluate task-specific and system-level performance. Evaluating generated actions in the unified environment reveals how local interventions benefit subsystems and affect system-level performance, which enables decision-making through an analysis–execution feedback loop.

4.2 Spatiotemporal Memory Management

4.2.1 Episodic Spatiotemporal Context Cache

Analytics from executable spatiotemporal reasoning are preserved within an episode using an Episodic Spatiotemporal Context Cache (ESCC), a structured working memory over spatial, temporal, and task dimensions that supports multi-step planning and reuse of analytical context.

During decision-making, the agent queries ESCC_{t-1} to retrieve historical analytics that can be utilized in the current reasoning episode. The retrieved context is then combined with the current analytical result Y_t^{ana} for action generation:

$$\mathcal{A}_t = \Phi_{\text{act}}(Y_t^{\text{ana}}, \text{Retrieve}(\text{ESCC}_{t-1}), \mathcal{T}'), \quad (11)$$

where $\text{Retrieve}(\cdot)$ denotes the retrieval process of the agent to select relevant cached analytics. ESCC therefore grounds decisions in both the current diagnosis and prior context within the same episode.

4.2.2 Procedural Spatiotemporal Memory

While ESCC supports data-grounded reasoning within a single episode, long-horizon optimization requires accumulating experience across episodes. We introduce a Procedural Spatiotemporal Memory (PSM) that distills episode-level experience into persistent procedural knowledge. Instead of storing raw intermediate results, PSM abstracts recurring traffic patterns, coordination strategies, and failure modes, enabling continual self-improvement of reasoning and control during iterative interactions.

We perform an episode-level summarization step that extracts higher-level procedural insights, including: (1) *recurring congestion patterns* (e.g., peak-direction spillback chains or station overload), (2) *effective system-level coordination strategies* (e.g., cross-subsystem optimization), and (3) *systematic failure modes* (e.g., repeated spillback caused by locally myopic interventions). Formally, the episode-level summarization is represented as:

$$Y_e^{\text{sum}} = \Phi_{\text{sum}}(\text{ESCC}_e, \mathcal{T}'), \quad (12)$$

where $\Phi_{\text{sum}}(\cdot)$ denotes the summarization process of the LLM agent for extracting procedural insights from reasoning episodes, capturing recurring traffic patterns, coordination strategies, and failure modes within interactive episodes.

All episodic summaries are maintained in the persistent Procedural Spatiotemporal Memory (PSM), which aggregates long-horizon experience across episodes. To prevent uncontrolled memory growth while preserving representative and informative knowledge, the agent performs memory management through selective insertion, merging, and pruning. Given the newly abstracted insight Y_e^{sum} from episode e , the PSM is updated as:

$$\text{PSM}_e = \Phi_{\text{update}}(\text{PSM}_{e-1}, Y_e^{\text{sum}}, \mathcal{T}'), \quad (13)$$

where $\Phi_{\text{update}}(\cdot)$ denotes the reasoning process to update PSM. Similar items are merged, obsolete or low-value entries are pruned, and informative patterns are retained as long-term procedural knowledge for stable coordinated control.

4.3 RL for System-Level Traffic Control

4.3.1 Supervised Multi-Task Cold-Start

To bootstrap generalizable reasoning across heterogeneous subsystems and traffic dynamics, we first train the agent using supervised learning on diverse interaction trajectories. Training scenarios cover:

(1) *spatial regions*, spanning regions with different traffic networks; (2) *subsystem task configurations*, including diverse suites of control tasks from heterogeneous traffic subsystems; and (3) *temporal dynamics*, capturing daily traffic patterns including peak and off-peak fluctuations.

Reference trajectories are generated using a generalist LLM (*e.g.*, DeepSeek (Liu et al., 2025)), producing high-quality interaction traces under coupled cross-subsystem dynamics. To bias learning toward globally beneficial behaviors, we retain only trajectories that outperform optimization-based baselines in system-level traffic efficiency (*i.e.*, global travel time). The LLM backbone is then trained via supervised fine-tuning to imitate these high-quality interactions, providing a strong policy prior and stabilizing early-stage exploration for subsequent RL training.

4.3.2 System-Aware Reward Construction

To enable unified optimization under heterogeneous and tightly coupled objectives, we design a system-level reward that encourages the agent to optimize overall traffic efficiency rather than isolated subsystem-specific outcomes:

$$R_{\text{env}}(s_t, \mathcal{A}_{t:t+H}) = \sum_{\tau=t}^{t+H} f_{\text{TP}}(s_{\tau}, a_{\tau}) - f_{\text{TT}}(s_{\tau}, a_{\tau}) + f_{\text{RI}}(s_{\tau}, a_{\tau}) \quad (14)$$

where $f_{\text{TP}}(\cdot)$, $f_{\text{TT}}(\cdot)$, $f_{\text{RI}}(\cdot)$ denote normalized throughput, average travel time, and relative improvement over the optimization-based baseline, respectively. These signals quantify whether control actions across heterogeneous traffic subsystems contribute to system-level traffic efficiency.

Quantitative metrics alone may not capture whether decisions are behaviorally coherent across subsystems. We further introduce an extra coordination reward, instantiated through LLM-as-a-judge (Tan et al., 2025), to evaluate cross-subsystem trade-offs, detect conflicting actions, and encourage alignment with system-level objectives $R_{\text{coord}}(s_t, \mathcal{A}_{t:t+H}, \mathcal{T}')$, which scores coordination quality, action consistency, and global traffic efficiency. The total reward is a weighted sum of system-level metrics and coordination evaluation:

$$R(s_t, \mathcal{A}_{t:t+H}, \mathcal{T}') = R_{\text{env}}(s_t, \mathcal{A}_{t:t+H}) + R_{\text{coord}}(s_t, \mathcal{A}_{t:t+H}, \mathcal{T}'). \quad (15)$$

4.3.3 System-Aligned Policy Optimization

We optimize the agent with agentic RL using Group Relative Policy Optimization (GRPO) (Guo et al.,

Table 1: Statistics of our studied regions, including road network scale and transportation demand dynamics.

Region	Road Network			Transportation Demand			
	Junctions	Lanes	Lane-km	Taxi	Public Transit	Walk	Total
Manhattan	10593	26329	1501.00	12111	51162	16816	98819
Queens	17896	50461	2672.99	3748	45711	13105	86486

2025). The agent iteratively interacts with the traffic environment, observes how interventions propagate across subsystems, and updates its policy to maximize system-level traffic efficiency.

Formally, the agent policy is defined as $\pi_{\theta}(\mathcal{A} | \mathcal{O})$. The agent’s objective is to maximize the expected cumulative system-level reward R . GRPO updates by computing a advantage function Adv_t and performing policy gradient:

$$\theta \leftarrow \theta + \eta \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(\mathcal{A} | \mathcal{O}) \cdot \text{Adv}_t], \quad (16)$$

Through multi-turn interaction, the agent internalizes how local interventions shape the global traffic state and learns robust system-level policies rather than isolated short-term gains.

5 Experiments

We conduct extensive experiments to evaluate TrafficClaw by answering these research questions:

- **RQ1:** How does TrafficClaw compare to the state-of-the-art traffic control agents in effectiveness and generalization ability?
- **RQ2:** How is TrafficClaw’s system-level optimization performance across subsystems?
- **RQ3:** How does each component of TrafficClaw improve its reasoning and decision-making?

5.1 Experimental Settings

Environment Settings: Our experimental regions include Manhattan, Queens, and Brooklyn in New York City, USA, three metropolitan regions with different spatial scales and demand dynamics (Table 1). We use SUMO (Alvarez Lopez et al., 2026) for simulation, covering subsystems of traffic signals, freeways, public transit, and taxi services. The simulation spans 24 hours, including rush hours (Adeyemi et al., 2021) from 6:00 am to 10:00 am and 3:00 pm to 8:00 pm. The details of selected tasks are summarized in Appendix C. The environment exposes 98 dynamic traffic-state features, 20 static data types, and 15 interaction APIs spanning subsystem and global operations. The details of the data interface are in Appendix A.

Baseline Methods: Baselines include classic methods (*e.g.*, Webster (Koonce et al., 2008)), RL-

Table 2: Experimental results across two NYC regions. Throughput (Thpt.), income, and completed trips (Trip) are higher-is-better ($\uparrow$), while travel time (Travel), fuel consumption (Fuel), queue length (Queue), waiting time (Wait), and electricity usage (Elec.) are lower-is-better ($\downarrow$).

Method/Model	Manhattan										Queens									
	in-domain					out-of-domain					in-domain					out-of-domain				
	Signal Control	Bus Sched.	Taxi Dispatching	Subway Sched.	Ramp Metering	Signal Control	Bus Sched.	Taxi Dispatching	Subway Sched.	Ramp Metering	Signal Control	Bus Sched.	Taxi Dispatching	Subway Sched.	Ramp Metering	Signal Control	Bus Sched.	Taxi Dispatching	Subway Sched.	Ramp Metering
	Thpt. ($\uparrow$)	Travel ($\downarrow$)	Fuel ($\downarrow$)	Wait ($\downarrow$)	Income ($\uparrow$)	Trip ($\uparrow$)	Elec. ($\downarrow$)	Wait ($\downarrow$)	Travel ($\downarrow$)	Queue ($\downarrow$)	Thpt. ($\uparrow$)	Travel ($\downarrow$)	Fuel ($\downarrow$)	Wait ($\downarrow$)	Income ($\uparrow$)	Trip ($\uparrow$)	Elec. ($\downarrow$)	Wait ($\downarrow$)	Travel ($\downarrow$)	Queue ($\downarrow$)
Classic Methods	122.07	481.80	339.69	512.11	6803.06	590	330.04	603.01	138.16	4.08	152.52	576.38	669.81	462.60	1925.61	300	185.11	761.33	148.21	11.46
RL-based Models	89.30	527.98	657.83	539.69	6485.59	610	231.32	340.26	189.81	4.86	100.12	654.18	1381.06	509.44	2116.13	282	170.71	242.62	297.08	12.40
LLM Agents																				
TrafficGPT	124.77	481.32	749.69	498.83	6901.45	725	314.76	678.61	102.99	4.29	158.50	570.57	1170.44	564.85	3451.62	371	218.45	391.12	296.66	12.73
SUMO-MCP	126.25	493.05	702.45	504.87	9463.35	727	315.41	294.61	106.20	3.53	159.65	564.06	903.04	471.71	2948.33	314	232.37	216.26	163.27	11.91
AIDE	126.46	491.50	257.58	511.95	7945.27	777	276.31	425.14	136.25	4.69	154.38	577.23	1042.66	449.21	3571.23	369	174.99	195.32	149.85	12.58
TrafficClaw with Generalist LLMs																				
DeepSeek-V3.2	121.98	493.48	927.15	419.42	7375.28	756	260.82	358.27	130.65	3.21	152.35	576.06	1668.97	458.92	3571.62	315	292.05	367.36	206.91	12.97
MiniMax-M2.5	127.04	493.84	1476.15	479.56	8849.25	767	291.00	324.92	124.93	4.07	160.02	566.78	2282.35	468.99	1652.83	329	155.75	722.67	153.07	11.75
Kimi-K2.5	126.25	497.24	1195.44	391.98	7752.26	536	250.58	357.27	140.50	3.34	152.33	572.12	2518.40	488.69	1921.63	320	249.85	312.08	95.09	13.23
Gemini-3.1-Pro	123.31	494.22	1006.75	398.29	7183.84	608	287.90	387.61	81.38	3.83	148.60	580.72	1880.37	441.01	1524.42	225	215.54	213.34	165.81	12.08
o4-mini	125.00	489.58	800.09	434.84	9787.97	837	352.94	259.82	103.29	3.80	166.13	567.16	996.86	499.39	3411.14	318	208.90	392.94	126.31	12.62
TrafficClaw with Base Model Family																				
Qwen3-8B	124.17	487.60	307.30	502.74	9201.60	781	317.08	350.92	99.77	3.76	138.67	584.25	686.50	507.52	3139.57	315	209.41	286.84	108.29	12.22
Qwen3-32B	121.13	488.68	295.91	532.29	6055.99	524	258.85	412.06	127.86	3.38	149.19	583.74	793.40	528.89	1645.00	248	194.89	313.98	141.16	12.38
Qwen3-235B	123.13	486.25	429.80	490.24	9327.90	873	270.22	427.45	108.75	3.68	150.73	575.61	980.78	494.84	1848.09	244	163.48	249.90	103.13	12.54
TrafficClaw (8B)	126.04	482.97	256.16	437.17	11017.49	1149	294.53	220.99	96.09	3.64	159.69	578.91	823.68	450.12	3781.19	486	194.23	177.03	100.51	14.16

Table 3: Transferability comparison in Brooklyn.

Method/Model	Signal Control		Bus Scheduling		Taxi Dispatching	
	Thpt. ($\uparrow$)	Travel ($\downarrow$)	Fuel ($\downarrow$)	Travel ($\downarrow$)	Income ($\uparrow$)	Trip ($\uparrow$)
Classic Method	160.71	509.55	226.34	231.18	748.30	99
RL-based Model	125.83	564.19	362.09	205.32	845.01	106
DeepSeek-V3.2	158.71	498.26	173.19	268.85	840.19	109
MiniMax-M2.5	161.21	503.46	171.65	311.37	726.02	99
Kimi-K2.5	160.38	505.76	148.92	273.30	775.26	104
o4-mini	161.04	518.26	172.89	315.83	675.75	88
TrafficClaw (8B)	160.25	500.26	150.83	238.07	976.76	123

based models (*e.g.*, CoLight (Wei et al., 2019a)), traffic LLM agents (TrafficGPT (Zhang et al., 2024), SUMO-MCP (Ye et al., 2025), and general data modeling agent AIDE (Jiang et al., 2025)), and leading LLMs (Qwen3 (Yang et al., 2025), DeepSeek-V3.2 (Liu et al., 2025), MiniMax-M2.5 (MiniMax AI, 2025), Kimi-K2.5 (Team et al., 2026), o4-mini (OpenAI, 2025), and Gemini-3.1-Pro (DeepMind, 2026)). Details are in Appendix B.

Training and Evaluation Settings: Qwen3-8B is used as the base model. In training, the agent is exposed to signal control, highway speed limit control, bus scheduling, and taxi dispatching, along with their corresponding cooperation settings. To assess generalization, ramp metering, subway scheduling, and their associated cooperative scenarios are held out for evaluation. Manhattan and Queens are used for training, while Brooklyn is reserved for evaluating cross-region transferability.

5.2 Overall Performance Comparison (RQ1)

Table 2 reports results on Manhattan and Queens under both in-domain and out-of-domain settings. Traditional optimization methods remain competitive on rule-driven objectives but struggle in heterogeneous subsystems due to limited adaptability

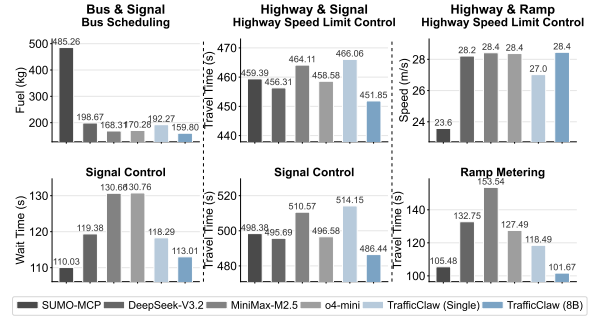


Figure 3: Multi-task cooperation comparison.

to complex urban dynamics. RL-based approaches require substantial task-specific engineering and scale poorly in city-wide environments with large state and action spaces. They only achieve competitive performance mainly on low-dimensional tasks, such as subway scheduling with only a few controllable lines. Traffic LLM agents (*i.e.*, TrafficGPT and SUMO-MCP) perform well on traffic signal control but degrade substantially on other subsystems, revealing limited cross-task generalization. In contrast, TrafficClaw consistently matches or surpasses leading LLM baselines despite using only an 8B backbone, while achieving more balanced trade-offs across conflicting objectives, such as reducing bus waiting time without excessive energy consumption. Table 3 further shows that TrafficClaw transfers effectively to Brooklyn while maintaining stable system-level performance.

5.3 Cross-Subsystem Coordination (RQ2)

We evaluate cooperative control on Manhattan across three cross-subsystem task suites: bus scheduling with signal control, highway speed limit control with signal control, and highway speed limit control with ramp metering over a 12-hour

Table 4: Average travel time (seconds) of all vehicles within the road network in Manhattan.

Method	Bus-Signal	Highway-Signal	Highway-Ramp
Classic Method	486.65	462.73	512.97
RL-based Model	499.10	456.17	528.23
TrafficClaw (Single)	465.95	459.71	505.04
TrafficClaw (w/o Co-Reward)	461.06	463.40	506.06
TrafficClaw	459.87	451.85	504.62

period. As shown in Figure 3, TrafficClaw consistently achieves the strongest joint performance by optimizing coupled subsystems rather than isolated tasks. In the bus-signal setting, it minimizes bus fuel consumption while maintaining near-optimal signal delay, indicating that coordinated signals reduce bus stopping and queuing. In the highway-related settings, TrafficClaw simultaneously improves highway efficiency, urban travel time, and ramp accessibility, whereas baselines typically optimize one objective at the expense of others.

Compared with TrafficClaw (Single), which optimizes each subsystem independently, cooperative TrafficClaw achieves more balanced gains across coupled tasks. Table 4 further demonstrates the importance of system-level coordination, with TrafficClaw consistently achieving the lowest global travel time. Furthermore, removing the coordination reward R_{coord} in TrafficClaw (w/o Co-Reward) leads to consistently higher travel time. This suggests that our proposed coordination rewards help resolve cross-subsystem trade-offs and mitigate conflicting actions. Overall, jointly optimizing coupled subsystems yields substantially stronger network-wide traffic efficiency.

5.4 Ablation Studies (RQ3)

As shown in Figure 4, supervised cold-start already improves substantially over the base model but remains unbalanced performance, indicating that imitation alone cannot capture cross-subsystem dynamics. RL-based tuning further refines policies through trial-and-error learning, yielding better system-level efficiency. Removing PSM causes systematic degradation across tasks, indicating that PSM is crucial for long-horizon adaptation and self-improvement. Appendix E further shows that PSM increasingly improves performance over time by accumulating reusable procedural knowledge through iterative interactions.

6 Related Work

LLM Agents: LLM agents extend language models with planning, tool use, environmental interaction, and episodic memory (Li et al., 2025b; Pang

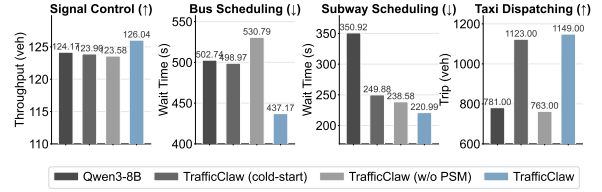


Figure 4: Ablation study on TrafficClaw.

et al., 2025; Dong et al., 2024; Steinberger and Contributors, 2025). Recent urban agents further introduce spatiotemporal reasoning (Feng et al., 2025; Li et al., 2025a; Lai et al., 2026), but they are mainly evaluated in loosely coupled settings rather than multi-subsystem traffic environments with shared physical constraints.

Traffic Control: Traffic control optimizes efficiency and sustainability (Tian et al., 2026, 2025) of urban traffic systems. It has evolved from classical optimization and RL toward LLM-based methods (Wei et al., 2019b; Siri et al., 2021; Farazi et al., 2021). LLM agents incorporate reasoning and flexible tool use for decision-making (Han et al., 2025). However, existing approaches remain largely task-specific and cannot effectively model coupled dynamics across heterogeneous traffic subsystems.

Agentic Reinforcement Learning: Agentic RL improves LLM decision-making through interaction, feedback, tool use, and long-horizon trajectories (Guo et al., 2025; Fang et al., 2025; Hu et al., 2025; Liang et al., 2024; Froger et al., 2025; Xi et al., 2025). However, most studies target digital environments, whereas urban traffic requires system-level optimization across heterogeneous and sometimes conflicting objectives.

7 Conclusion

We introduce TrafficClaw as a step toward physically grounded LLM agents for urban systems. Instead of treating traffic control as isolated tasks, TrafficClaw embeds agents within a unified physical traffic environment where shared infrastructure, mobility demand, and local interventions jointly shape network-wide dynamics. By integrating executable spatiotemporal reasoning, persistent memory, and system-level agentic RL, TrafficClaw enables long-horizon coordinated control across heterogeneous subsystems. Our results suggest that physical-world LLM agents require not only reasoning abilities, but also a unified environment that exposes coupled dynamics and feedback-driven optimization for system-level coordination.

Limitations

TrafficClaw focuses on unified agentic traffic control within large-scale urban environments. While the framework demonstrates strong generalization across regions, subsystems, and unseen task settings, several promising directions remain for future work. First, extending TrafficClaw to real-world deployment with online traffic streams, connected infrastructure, and human-in-the-loop operations would further validate its practicality under dynamic urban conditions. Second, future research may incorporate richer mobility behaviors, long-term traveler adaptation, and additional urban objectives such as emergency response and accessibility. Finally, integrating stronger safety constraints, uncertainty-aware reasoning, and more robust memory and reward mechanisms could further improve the reliability and scalability of long-horizon autonomous traffic control.

Ethical Considerations

TrafficClaw is designed to support more efficient and coordinated urban mobility management. Our experiments rely on aggregate, publicly available traffic and mobility data and do not require personally identifiable information. Nevertheless, traffic-control systems can influence congestion, accessibility, and service quality across different regions and populations. Real-world deployment should therefore include evaluation of fairness, environmental impact, and robustness under diverse operating conditions. We view TrafficClaw as a research and decision-support framework rather than a replacement for transportation authorities. Practical deployment should incorporate human oversight, transparent logging, safety constraints, and conservative rollout procedures to ensure accountable and reliable operation in real-world urban systems.

References

- Oluwaseun John Adeyemi, Ahmed A Arif, and Rajib Paul. 2021. Exploring the relationship of rush hour period and fatal and non-fatal crash injuries in the us: A systematic review and meta-analysis. *Accident Analysis & Prevention*, 163:106462.
- Pablo Alvarez Lopez, Angelo Banse, Michael Behrisch, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Ronald Nippold, and Peter Wagner. 2026. [Simulation of urban mobility \(sumo\)](#).
- James E Anderson. 2011. The gravity model. *Annu. Rev. Econ.*, 3(1):133–160.
- Francois Belletti, Daniel Haziza, Gabriel Gomes, and Alexandre M Bayen. 2017. Expert level control of ramp metering based on multi-task deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, 19(4):1198–1207.
- Nicolas Bougie and Narimawa Watanabe. 2025. Citysim: Modeling urban behaviors and city dynamics with large-scale llm-driven agent simulation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 215–229.
- Google DeepMind. 2026. [Gemini](#). Accessed: 2026-02-09.
- Yubo Dong, Xukun Zhu, Zhengzhe Pan, Linchao Zhu, and Yi Yang. 2024. Villageragent: A graph-based multi-agent framework for coordinating complex task dependencies in minecraft. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 16290–16314.
- Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, and 1 others. 2025. Towards general agentic intelligence via environment scaling. *arXiv preprint arXiv:2509.13311*.
- Nahid Parvez Farazi, Bo Zou, Tanvir Ahamed, and Limon Barua. 2021. Deep reinforcement learning in transportation research: A review. *Transportation research interdisciplinary perspectives*, 11:100425.
- Jie Feng, Yuwei Du, Jie Zhao, and Yong Li. 2025. Agentmove: A large language model based agentic framework for zero-shot next location prediction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1322–1338.
- Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, and 1 others. 2025. Are: Scaling up agent environments and evaluations. *arXiv preprint arXiv:2509.17158*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638.
- Jindong Han, Yansong Ning, Zirui Yuan, Hang Ni, Fan Liu, Tengfei Lyu, and Hao Liu. 2025. Large language model powered intelligent urban agents: Concepts, capabilities, and applications. *arXiv preprint arXiv:2507.00914*.
- Mengkang Hu, Pu Zhao, Can Xu, Qingfeng Sun, Jianguang Lou, Qingwei Lin, Ping Luo, and Saravan Rajmohan. 2025. Agentgen: Enhancing planning abilities for large language model based agent via environment and task generation. In *Proceedings of*

- the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 496–507.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. 2025. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*.
- Jiarui Jin, Ming Zhou, Weinan Zhang, Minne Li, Zilong Guo, Zhiwei Qin, Yan Jiao, Xiaocheng Tang, Chenxi Wang, Jun Wang, and 1 others. 2019. Coride: joint order dispatching and fleet management for multi-scale ride-hailing platforms. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1983–1992.
- Peter Koonce and 1 others. 2008. Traffic signal timing manual. Technical report, United States. Federal Highway Administration.
- Siqi Lai, Yansong Ning, Zirui Yuan, Zhixi Chen, and Hao Liu. 2026. USTBench: Benchmarking and dissecting spatiotemporal reasoning capabilities of LLMs as urban agents. In *The Fourteenth International Conference on Learning Representations*.
- Siqi Lai, Zhao Xu, Weijia Zhang, Hao Liu, and Hui Xiong. 2025. Llmlight: Large language models as traffic signal control agents. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 2335–2346.
- Wenbin Li, Di Yao, Ruibo Zhao, Wenjie Chen, Zijie Xu, Chengxue Luo, Chang Gong, Quanliang Jing, Haining Tan, and Jingping Bi. 2025a. Stbench: Assessing the ability of large language models in spatiotemporal analysis. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 749–752.
- Xiaoxi Li, Jiajie Jin, Guanting Dong, Hongjin Qian, Yongkang Wu, Ji-Rong Wen, Yutao Zhu, and Zhicheng Dou. 2025b. Webthinker: Empowering large reasoning models with deep research capability. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Zhibin Li, Pan Liu, Chengcheng Xu, Hui Duan, and Wei Wang. 2017. Reinforcement learning-based variable speed limit control strategy to reduce traffic congestion at freeway recurrent bottlenecks. *IEEE transactions on intelligent transportation systems*, 18(11):3204–3217.
- William Liang, Sam Wang, Hung-Ju Wang, Osbert Bastani, Dinesh Jayaraman, and Yecheng Jason Ma. 2024. Environment curriculum generation via large language models. In *8th Annual Conference on Robot Learning*.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, and 1 others. 2025. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.
- Metropolitan Transportation Authority (MTA). 2026. Mta open data program and transit schedules. <https://www.mta.info/open-data>. Public transit schedules and operational transit data provided by local transportation authorities.
- MiniMax AI. 2025. *Minimax m2 & agent: Ingenious in simplicity*. Accessed: 2026-01-08; MiniMax M2 is an open-source AI model optimized for coding and agentic workflows, offering efficient performance and high reasoning capability at low cost.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*.
- New York City Department of Transportation. 2022. Citywide mobility survey. <https://www.nyc.gov/html/dot/html/about/citywide-mobility-survey.shtml>.
- New York City Taxi & Limousine Commission. 2026. Nyc taxi and limousine commission (tlc) trip record data. <https://registry.opendata.aws/nyc-tlc-trip-records-pds/>. Yellow and green taxi trip records including pick-up/drop-off and related fields.
- OpenAI. 2025. *Introducing openai o3 and o4-mini*. Announcement of OpenAI’s o3 and o4-mini models with advanced reasoning and tool usage capabilities.
- OpenStreetMap contributors. 2026. *Openstreetmap planet data*. Accessed: 2026-01-24.
- Xianghe Pang, Shuo Tang, Rui Ye, Yuwen Du, Yaxin Du, and Siheng Chen. 2025. Browsermaster: Towards scalable web browsing via tool-augmented programmatic agent pair. *arXiv preprint arXiv:2508.09129*.
- Markos Papageorgiou, Habib Hadj-Salem, Jean-Marc Blosseville, and 1 others. 1991. Alinea: A local feedback control law for on-ramp metering. *Transportation research record*, 1320(1):58–67.
- Jinghua Piao, Yuwei Yan, Jun Zhang, Nian Li, Junbo Yan, Xiaochong Lan, Zhihong Lu, Zhiheng Zheng, Jing Yi Wang, Di Zhou, and 1 others. 2025. Agentsocty: Large-scale simulation of llm-driven generative agents advances understanding of human behaviors and society. *arXiv preprint arXiv:2502.08691*.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*.
- Silvia Siri, Cecilia Pasquale, Simona Saccone, and Antonella Ferrara. 2021. Freeway traffic control: A survey. *Automatica*, 130:109655.

- Peter Steinberger and OpenClaw Contributors. 2025. Openclaw: Your own personal ai assistant. any os. any platform. <https://github.com/openclaw/openclaw>. Accessed: 2026-04-09.
- SUMO Development Team. 2026. Taxi - sumo documentation. <https://sumo.dlr.de/docs/Simulation/Taxi.html>. Accessed: 2026-04-12.
- Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Tang, Alejandro Cuadron, Chenguang Wang, Raluca Popa, and Ion Stoica. 2025. Judgebench: A benchmark for evaluating llm-based judges. In *International Conference on Learning Representations*, volume 2025, pages 63277–63303.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, and 1 others. 2026. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*.
- Jindong Tian, Yifei Ding, Ronghui Xu, Hao Miao, Chenjuan Guo, and Bin Yang. 2026. Arrow: An adaptive rollout and routing method for global weather forecasting.
- Jindong Tian, Yuxuan Liang, Ronghui Xu, Peng Chen, Chenjuan Guo, Aoying Zhou, Lujia Pan, Zhongwen Rao, and Bin Yang. 2025. Air quality prediction with physics-guided dual neural odes in open systems.
- Jiawei Wang and Lijun Sun. 2022. Robust dynamic bus control: A distributional multi-agent reinforcement learning approach. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):4075–4088.
- Zhaoyuan Wang, Zheyi Pan, Shun Chen, Shenggong Ji, Xiuwen Yi, Junbo Zhang, Jingyuan Wang, Zhiguo Gong, Tianrui Li, and Yu Zheng. 2022. Shortening passengers’ travel time: A dynamic metro train scheduling approach using deep reinforcement learning. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):5282–5295.
- Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinsi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu, and Zhenhui Li. 2019a. Colight: Learning network-level cooperation for traffic signal control. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1913–1922.
- Hua Wei, Guanjie Zheng, Vikash Gayah, and Zhenhui Li. 2019b. A survey on traffic signal control methods. *arXiv preprint arXiv:1904.08117*.
- Yuankai Wu, Huachun Tan, Lingqiao Qin, and Bin Ran. 2020. Differential variable speed limits control for freeway recurrent bottlenecks via deep actor-critic algorithm. *Transportation research part C: emerging technologies*, 117:102649.
- Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, and 1 others. 2025. Agentgym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27914–27961.
- Qian Xu, Zhiguang Liu, and Zhigang Xu. 2022. A novel ramp metering algorithm based on deep reinforcement learning. In *2022 2nd International Conference on Algorithms, High Performance Computing and Artificial Intelligence (AHPCAI)*, pages 128–133. IEEE.
- Ronghui Xu, Jihao Chen, Jindong Tian, Chenjuan Guo, and Bin Yang. 2026. Most: A foundation model for multi-modality spatio-temporal traffic prediction. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 1716–1727.
- Ronghui Xu, Hanyin Cheng, Chenjuan Guo, Hongfan Gao, Jilin Hu, Sean Bin Yang, and Bin Yang. 2025. Mm-path: Multi-modal, multi-granularity path representation learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 1703–1714.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Chenglong Ye, Gang Xiong, Junyou Shang, Xingyuan Dai, Xiaoyan Gong, and Yisheng Lv. 2025. Sumo-mcp: Leveraging the model context protocol for autonomous traffic simulation and optimization. *arXiv preprint arXiv:2506.03548*.
- Zirui Yuan, Siqi Lai, and Hao Liu. 2025. Collm-light: Cooperative large language model agents for network-wide traffic signal control. *arXiv preprint arXiv:2503.11739*.
- Siyao Zhang, Daocheng Fu, Wenzhe Liang, Zhao Zhang, Bin Yu, Pinlong Cai, and Baozhen Yao. 2024. Trafficgpt: Viewing, processing and interacting with traffic foundation models. *Transport Policy*, 150:95–105.
- Dong Zhao, Adriana-Simona Mihăiță, Yuming Ou, Hanna Grzybowska, and Mo Li. 2024. Origin–destination matrix estimation for public transport: A multi-modal weighted graph approach. *Transportation Research Part C: Emerging Technologies*, 165:104694.

Table 5: Detailed dynamic traffic-state features.

Category	Feature Names	Count	Description
Lane-level Features	queue_length, queue_density, moving_vehicles, average_speed, average_waiting_time, cell_occupancy, lane_density, throughput_potential, occupancy, halting_number, max_speed, arrival_rate, entering_vehicles, vehicle_count, vehicle_details (speed, position, waiting_time)	15	Lane-level traffic-state features returned by read_lane_traffic_states.
Highway Features	segment_speed, segment_density, segment_occupancy, segment_speed_limit, segment_default_speed_limit, segment_congestion_ratio, segment_speed_ratio, segment_speed_pressure, road_speed, road_density, road_occupancy, current_speed_limits, default_speed_limits	13	Freeway segment-level traffic-state features returned by read_highway_traffic_states.
Ramp Features	vehicle_count, queue_length, queue_density, moving_vehicles, average_speed, average_waiting_time, cell_occupancy, lane_density, occupancy, halting_number, max_speed, arrival_rate, lane_length, road_id, direction, start_intersection, end_intersection	17	Ramp-lane traffic-state features returned by read_ramp_lane_traffic_states.
Bus Features	active_buses, headway, station_count, departure_time, travel_time, current_edge, speed, passenger_count, capacity, load_ratio, next_station, next_station_dwell_time, waiting_count, avg_waiting_time, max_waiting_time, waiting_time_distribution (0–60s, 60–180s, 180–300s, >300s)	16	Bus operation features returned by read_bus_states.
Subway Features	active_trains, headway, station_count, departure_time, travel_time, current_edge, speed, passenger_count, capacity, load_ratio, next_station, next_station_dwell_time, waiting_count, avg_waiting_time, max_waiting_time, waiting_time_distribution (0–60s, 60–180s, 180–300s, >300s)	16	Subway operation features returned by read_subway_states.
Taxi Features	fleet_size, idle_count, pickup_count, occupied_count, utilization_rate, pending_reservations, taxi_state, customers, current_edge, current_taz, position, speed, cumulative_income, recent_order_count	14	Taxi fleet features returned by read_taxi_traffic_states and taxi_fleet_state.
Global Features	total_vehicles, avg_queue_length, avg_speed, avg_waiting_time, congestion_level, intersection_count, lane_count	7	Global aggregated features returned by analyze_zone_traffic.

A Observation and Interaction Interfaces

Table 5-7 shows the dynamic traffic-state features, static data, and interaction APIs given to the agent.

B Evaluated Baselines

We summarize the baselines used in our comparisons below, moving the detailed descriptions out of the main text while keeping enough context to interpret their roles in the experiments. All LLM-based agents and baselines are provided with the same environment interaction interfaces as TrafficClaw for fair comparison. The training settings of RL-based models, including reward functions, hyperparameters, and input features, follow the configurations reported in the original papers.

- **Classic Method:** For classic methods, we use Webster (Koonce et al., 2008) for signal control; a greedy heuristic (SUMO Development Team, 2026) that drives taxis to the nearest pickup location for taxi dispatching; ALINEA (Papageorgiou et al., 1991) for ramp metering; the default policies published by NYC (Metropolitan Transportation Authority (MTA), 2026; OpenStreetMap contributors, 2026; New York City Department of Transportation, 2022) for bus and subway scheduling, highway speed limit control.
- **RL-based Model:** For RL-based models, we use CoLight (Wei et al., 2019a) for signal control;

DQN (Mnih et al., 2013) for taxi dispatching; MARL (Wang and Sun, 2022) for bus scheduling; AutoDwell (Wang et al., 2022) for subway scheduling; DDQN (Xu et al., 2022) for ramp metering; actor-critic RL (Wu et al., 2020) for highway speed limit control. Same as TrafficClaw, we use Manhattan and Queens for training, while Brooklyn is reserved for evaluating cross-region transferability. Epoch is set to 100, batch size is set to 64, learning rate is set to 0.0003, and reward discount factor is set to 0.95 for all tasks.

- **TrafficGPT (Zhang et al., 2024)** is a traffic LLM agent that combines natural-language reasoning with traffic analytics tools for decision support.
- **SUMO-MCP (Ye et al., 2025)** is an autonomous traffic simulation agent built on the Model Context Protocol, enabling tool calling for scenario generation, simulation execution, and strategy comparison in SUMO.
- **AIDE (Jiang et al., 2025)** is a general coding agent that searches over executable programs, making it a strong baseline for data-driven modeling and optimization.
- **Qwen3-Series (Yang et al., 2025)** (8B, 32B, 235B) is developed by Alibaba Cloud, serves as the foundational open-source backbone in our training pipeline. We include it to assess the improvements achieved through our environment

Table 6: Detailed static traffic-state data.

Data Resource Name	Type	Description
zone_dict	dict	Zone infrastructure dictionary containing lanes, intersections, highways, ramps, and transit stations.
lane_dict	dict	Lane metadata dictionary containing location, direction, lane group, and related attributes.
lane_inter_graph	NetworkX DiGraph	Directed graph connecting lane groups and intersections.
highway_segment_graph	NetworkX DiGraph	Directed graph describing connectivity among freeway segments.
highway_segment_dict	dict	Freeway segment metadata dictionary.
ramp_lane_graph	NetworkX DiGraph	Directed graph describing ramp-lane topology and upstream/downstream relations.
transit_graph	NetworkX DiGraph	Public transit network graph covering bus and subway routes and stations.
zone_graph	NetworkX DiGraph	Directed graph describing zone adjacency relations.
network_graphs	dict	Base-layer network graphs, including lane_graph, road_graph, and transit_graph.
network_dicts	dict	Base-layer metadata dictionaries, including lane_dict, road_dict, and station_dict.
bus_route_info	dict	Detailed bus route information, including static structure and runtime operation data.
current_signal_config	dict	Current signal control configuration.
current_highway_speed_limit_config	dict	Current freeway speed-limit control configuration.
current_ramp_metering_config	dict	Current ramp-metering control configuration.
current_bus_schedule	dict	Current bus scheduling configuration.
current_subway_schedule	dict	Current subway scheduling configuration.
current_taxi_config	dict	Current taxi dispatch and fleet-management configuration.
taxi_fleet_state	dict	Current taxi fleet state.
pending_reservations	dict	Pending taxi reservation requests awaiting assignment.
taz_stats	dict	TAZ-level statistics including demand, supply, and matching rate.

scaling and agentic reinforcement learning, rather than the backbone alone.

- **DeepSeek-V3.2** (Liu et al., 2025) is an open-source generalist model that supports adaptive reasoning, which achieves competitive performance in reasoning and coding tasks.
- **MiniMax-M2.5** (MiniMax AI, 2025) is an open-source LLM for long-horizon agentic tasks.
- **Kimi-K2.5** (Team et al., 2026) is from Moonshot AI with strong reasoning and autonomous problem-solving capabilities.
- **o4-mini** (OpenAI, 2025) is from OpenAI with strong tool-use and deliberate reasoning capabilities, offering an efficiency-oriented proprietary baseline.
- **Gemini-3.1-Pro** (DeepMind, 2026) is a proprietary multimodal reasoning model with strong long-context and coding capabilities, included as another SOTA generalist baseline.

C Tasks and Evaluation Metrics

We summarize each evaluated task, including its goal in traffic control and the evaluation metrics, in the following list.

- **Signal control (traffic signals).** The agent optimizes signal phases by adjusting signal cycle time and phase durations to improve efficiency, evaluated by throughput (veh/h), average waiting time (s), and average travel time (s).

- **Highway speed limit control (freeways).** The agent adjusts variable speed limits to regulate mainline flow and avoid congestion, evaluated by average travel time and average speed (m/s).
- **Ramp metering (freeways).** The agent meters ramp by adjusting ramp open duration to protect mainline conditions while limiting ramp-side congestion, evaluated by average travel time and average queue length.
- **Bus scheduling (public transit).** The agent adjusts bus departures by adjusting bus stop dwell time and frequency to balance service reliability and operational cost, evaluated by fuel consumption (kg) and passenger waiting time (s).
- **Subway scheduling (public transit).** The agent adjusts subway departures by adjusting subway stop dwell time and frequency to balance passenger service and energy use, evaluated by electricity consumption (kWh) and waiting time.

D Agentic RL Convergence Analysis

We analyze the agentic RL convergence during training. Figure 5 plots the average reward per episode. The reward stabilizes during training, indicating the effectiveness of our training.

E Memory Accumulation

Figure 6 shows the benefits of memory become increasingly pronounced over time as the agent ac-

Table 7: Detailed callable interaction APIs.

Function Name	Function Description
read_bus_states	Query historical bus operation states, including vehicle conditions and station waiting information.
read_highway_traffic_states	Query historical freeway traffic states, including speed, density, and occupancy.
predict_arima	Perform ARIMA-based time-series forecasting for future traffic flow patterns (<i>e.g.</i> , future traffic patterns or downstream lane occupancy).
analyze_zone_traffic	Analyze zone- or TAZ-level traffic conditions by aggregating lane-level indicators.
identify_congestion_hotspots	Identify congestion hotspots in the network based on queue-length and speed thresholds.
calculate_network_metrics	Compute network-level traffic indicators, such as congestion index, average speed, and throughput potential.
read_ramp_lane_traffic_states	Query historical ramp-lane traffic states, including occupancy and queue length.
read_lane_traffic_states	Query historical lane traffic states, including queue length, speed, and waiting time.
read_subway_states	Query historical subway operation states, including train conditions and station waiting information.
read_taxi_traffic_states	Query historical taxi traffic states, including fleet utilization and pending orders.
dispatch_taxi	Construct taxi dispatch decisions by assigning idle taxis to pending reservations.
reposition_taxi	Construct taxi repositioning decisions by moving idle taxis to target zones.
rank_idle_taxis_by_distance	Rank idle taxis by Euclidean distance for distance-aware dispatching.
get_zone_infrastructure	Query infrastructure information for a target zone.
get_zones_by_infrastructure	Retrieve zones containing specified infrastructure elements.

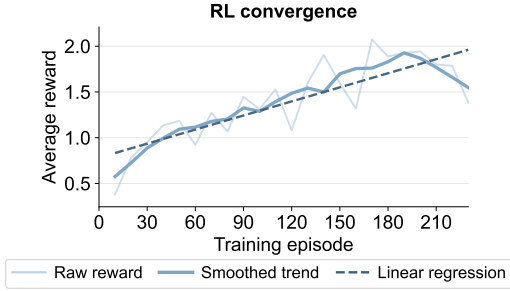


Figure 5: RL convergence of the agent during training.

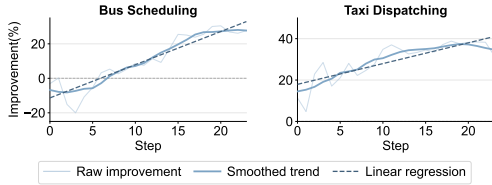


Figure 6: Improvement in agent control performance over time compared to TrafficClaw (w/o PSM).

cumulates experience through iterative interactions. This indicates that PSM serves as a mechanism for consolidating observed traffic dynamics and effective control strategies into reusable procedural knowledge. The key insight is that memory helps the agent accumulate, reuse, and refine effective coordination routines, thereby supporting self-improvement across traffic regimes and tasks.

F Case Study

Figure 7 presents two representative Manhattan control episodes that illustrate how PSM converts prior interaction experience into reusable procedu-

Table 8: Confidence interval of TrafficClaw.

Metric	Signal Control	Bus-Signal	Highway-Ramp
Travel Time	482.97 (± 3.74)	459.87 (± 5.09)	504.62 (± 2.08)
Waiting Time	133.84 (± 0.99)	113.01 (± 2.80)	413.38 (± 6.26)

ral knowledge. PSM stores structured coordination patterns that capture cross-subsystem dependencies, regime-specific strategies, and historical trade-offs. In the first case, the agent retrieves knowledge about the coupling between signal control and bus scheduling under low-demand conditions, enabling coordinated optimization across both subsystems. In the second case, PSM recalls night-period traffic patterns and guides conservative signal adjustments suited to low-volume dynamics. In both cases, the agent improves stability and decision quality by reusing accumulated experience rather than relying solely on reactive reasoning, demonstrating continual self-improvement through procedural memory accumulation and adaptation.

G Confidence Interval of Evaluation

Table 8 reports the confidence intervals of TrafficClaw computed from three independent experimental runs with different random seeds on SUMO, covering both single-subsystem and multi-subsystem cooperative settings.

H Experiment Configuration

We train TrafficClaw using VeRL (Sheng et al., 2024) and evaluate both open-source and propri-

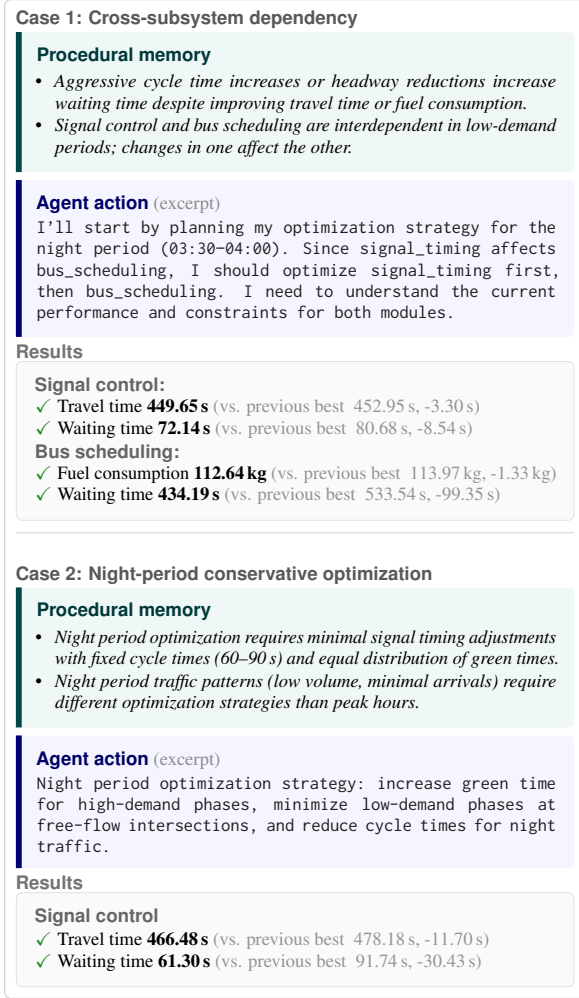


Figure 7: Case study of self-improvement.

etary LLMs through the SiliconFlow and OpenAI APIs, respectively. All evaluations use a fixed decoding temperature of 0.0 to ensure deterministic and reproducible results. The reference trajectories for multi-task cold-start are generated by the DeepSeek-V3.2. The decision horizon for all tasks is set to 30 minutes. TrafficClaw supports city-scale deployment on a single NVIDIA A800 GPU, requiring approximately 10.37s for each reasoning episode and 114.11s for each rollout.

I License

Our code and the post-trained TrafficClaw model are released under the MIT License. For third-party models, Qwen3-Series (Yang et al., 2025) is licensed under Apache-2.0, DeepSeek-V3.2 (Liu et al., 2025) under MIT, and MiniMax-M2.5 (MiniMax AI, 2025) and Kimi-K2.5 (Team et al., 2026) under Modified MIT licenses; o4-mini (OpenAI, 2025) and Gemini-3.1-Pro (DeepMind, 2026) are proprietary API models used under the correspond-

ing OpenAI and Google service terms. For third-party code and tools, VeRL (Sheng et al., 2024) is Apache-2.0, AIDE (Jiang et al., 2025) is MIT, TrafficGPT (Zhang et al., 2024) is GPL-3.0, and SUMO (Alvarez Lopez et al., 2026) is EPL-2.0 with GPL v2-or-later as a secondary license option; when a baseline repository (Wei et al., 2019a; Mnih et al., 2013; Wang and Sun, 2022; Wang et al., 2022; Xu et al., 2022; Wu et al., 2020) does not declare an explicit license, we use it only for evaluation and do not redistribute its code. For data, NYC Mobility Survey (New York City Department of Transportation, 2022), and NYC TLC trip records (New York City Taxi & Limousine Commission, 2026) are public NYC Open Data resources governed by NYC terms of use, OpenStreetMap data (OpenStreetMap contributors, 2026) is licensed under ODbL-1.0, and MTA schedule/open-data feeds (Metropolitan Transportation Authority (MTA), 2026) are used under the MTA data feed terms and conditions. These existing artifacts are used consistently with their stated intended use: open-weight models are used as research baselines or training backbones under their model licenses, proprietary models are accessed only through their official APIs, third-party code and simulators are used for training or evaluation under their software licenses, and public transportation datasets are used for research simulation and evaluation under the corresponding data-provider terms.

J Data Anonymization

The traffic demand data used in our experiments is sourced from publicly available NYC Open Data (New York City Department of Transportation, 2022; New York City Taxi & Limousine Commission, 2026). The dataset is anonymized and does not contain any personally identifiable information regarding drivers or passengers.

K AI Assistant Usage

We used AI assistants to polish the writing of this paper and to improve the visual presentation of figures. The figure-related assistance was limited to artistic styling and layout refinement; all underlying data, quantitative values, experimental results, and plots are obtained from the original outputs.

L Agent Prompt

We provide the prompt used in TrafficClaw and example LLM outputs below.

System prompt

Urban Transportation Joint Control Agent

Your Role

You are an expert urban transportation control agent optimizing multiple systems jointly. Your goal is to coordinate multiple control modules to improve overall urban mobility.

Global Knowledge

Enabled Modules

- signal_timing
- bus_scheduling

(Only these modules can be optimized.)

Cross-Module Dependencies

- signal_timing: affects: bus_scheduling
- bus_scheduling: affected_by: signal_timing

Shared Optimization Principles

- Optimize upstream modules first (modules with no dependencies)
- Downstream modules should consider upstream module decisions
- Coordinate related modules for better overall performance

Turn Limit

You have a maximum of 20 dialogue turns. Each action counts as one turn. Plan efficiently and use FINISH when complete.

Available Actions

1. **PLAN:** Think and plan your optimization strategy
2. **GET_CONTROL_API:** Query module-specific APIs, data, and domain knowledge
Available modules: signal_timing, bus_scheduling
3. **DATA_ANALYSIS:** Analyze traffic data (use save_cache() to store results)
4. **POLICY_PLANNING:** Design control configurations (simulation runs automatically)
5. **DEBUG:** Fix code errors
6. **FINISH:** Complete optimization

Previous Optimization Experience (Memory)

1. Conservative optimization (5–10% changes) is more effective than aggressive changes for both signal timing and bus scheduling.
2. Targeted adjustments (top 50 intersections, top 30 routes) work better than broad changes, focusing on queue > 30 or waiting time > 80 thresholds.
3. Signal timing optimization should precede bus scheduling optimization due to dependency relationships.
4. Signal timing improvements often come with waiting time trade-offs but can maintain throughput.
5. Bus scheduling improvements can increase fuel consumption but reduce passenger waiting time significantly.
6. Accept 1–2% reward decrease if specific metrics (like passenger waiting time) improve significantly.
7. Use 85–90% headway reductions for bus scheduling adjustments based on waiting time thresholds (> 150 s).
8. Midday period (12:30–13:00) shows consistent traffic patterns suitable for targeted optimization.
9. Focus on intersections with queue > 30 or waiting time > 80 for signal timing adjustments.
10. Network has complex intersections with multiple phase types requiring careful signal timing adjustments.

Important Notes

- Use GET_CONTROL_API to query module APIs and domain knowledge before optimization
- Only enabled modules can be optimized
- Consider module dependencies when planning optimization order
- Simulation is automatically executed after POLICY_PLANNING

Required Formats (Critical)

- Always start with: ACTION: <ACTION_NAME>
- GET_CONTROL_API format, DATA_ANALYSIS format, POLICY_PLANNING format, DEBUG (policy) format (see listing below)
- One action per turn. Do not combine actions in a single response.

Required action format examples (verbatim from the system prompt).

GET_CONTROL_API format:

ACTION: GET_CONTROL_API

Module: signal_timing, bus_scheduling # comma-separated allowed

DATA_ANALYSIS format:

ACTION: DATA_ANALYSIS

Use save_cache(...) to store analysis results

POLICY_PLANNING format:

ACTION: POLICY_PLANNING

Control Modules: signal_timing, bus_scheduling

Define a variable named 'config' and do NOT use return

config = {...}

DEBUG (policy) format:

ACTION: DEBUG

Debug Type: POLICY_PLANNING

Control Modules: signal_timing, bus_scheduling

Fixed code

Begin with PLAN to outline your optimization strategy, then use GET_CONTROL_API to query module APIs.

Agent Response: Executable Spatiotemporal Analysis

Agent response (abbreviated turn)

ACTION: DATA_ANALYSIS

```
# DATA_ANALYSIS (abbreviated): joint signal_timing + bus_scheduling analytics
print("=== Signal Timing Analysis ===")
signal_data = read_lane_traffic_states()
if 'error' not in signal_data:
    snapshots = signal_data.get('snapshots', [])
    if snapshots:
        lane_states = snapshots[-1].get('lane_states', {})
        intersection_queues, phase_demands = {}, {}
        for lane_id, lane_data in lane_states.items():
            q = lane_data.get('queue_length', 0)
            end_inter = lane_data.get('end_intersection')
            loc_dir = lane_data.get('loc_dir', '')
            if end_inter and end_inter != 'None':
                intersection_queues.setdefault(end_inter, []).append(q)
            if loc_dir and end_inter and end_inter != 'None':
                key = (end_inter, loc_dir)
                phase_demands[key] = phase_demands.get(key, 0) + q

        intersection_avg_queue = {
            i: sum(qs) / len(qs) for i, qs in intersection_queues.items() if qs
        }
        busy = [i for i, avg in intersection_avg_queue.items() if avg >= 5]
        free = [i for i, avg in intersection_avg_queue.items() if avg < 5]
        save_cache({
            'busy_intersections': busy,
            'free_flow_intersections': free,
            'phase_demands': phase_demands,
            'intersection_avg_queue': intersection_avg_queue,
        })

print("\n=== Bus Scheduling Analysis ===")
bus_data = read_bus_states()
if 'error' not in bus_data:
    snapshots = bus_data.get('snapshots', [])
    if snapshots:
        bus_lines = snapshots[-1].get('bus', {}).get('lines', {})
        station_waiting_analysis, route_headway_analysis = {}, {}
        save_cache({'route_headway_analysis': route_headway_analysis}) # ... stations / routes omitted

print("\nData analysis complete. Results saved to cache.")
```

Agent Response: Feedback-Driven decision-making

Agent response (abbreviated turn)

ACTION: POLICY_PLANNING

Control Modules: signal_timing, bus_scheduling

```
# POLICY_PLANNING (abbreviated): write executable configs for enabled modules
import copy
signal_config = current_signal_config.copy()
phase_demands_cache = load_cache('phase_demands')
for inter_id, phases in signal_config.items():
    current_phases = current_signal_config[inter_id]
    phase_demand_map = {}
    if phase_demands_cache and f'phase_demand_{inter_id}' in phase_demands_cache:
        phase_demand_map = phase_demands_cache[f'phase_demand_{inter_id}']
    total_demand = sum(phase_demand_map.values())
    if total_demand > 0:
        new_durations = {}
        for phase_name, dur in current_phases.items():
            d = phase_demand_map.get(phase_name, 0)
            new_durations[phase_name] = max(10, int(dur * (d / total_demand))) if d > 0 else 10
        signal_config[inter_id] = new_durations # ... cycle-time scaling omitted
    else:
        signal_config[inter_id] = {p: max(10, current_phases[p]) for p in current_phases}

bus_config = {}
for route_id, config in current_bus_schedule.items():
    bus_config[route_id] = {"timetable": []}
    for period in config['timetable']:
        new_period = {
            "time_range": period["time_range"],
            "headway": period["headway"],
            "schedule": copy.deepcopy(period["schedule"]),
        }
        bus_config[route_id]["timetable"].append(new_period) # ... dwell/headway edits omitted
config = {"signal_timing": signal_config, "bus_scheduling": bus_config}
```

Procedural Spatiotemporal Memory Update Prompt

[You have just completed an optimization session. Above is the complete conversation history including all your actions, code executions, simulation results, and feedback.]

Session Summary (for quick reference, but review full conversation above):

- Total turns: 13
- Actions taken: PLAN(4), GET_CONTROL_API(1), DATA_ANALYSIS(3), POLICY_PLANNING(3), DEBUG(1), FINISH(1)

Best Module-level Results:

- Signal timing (best turn 5): reward 218,246.00; avg queue 8.43; avg waiting 59.87 s; throughput 32.00.
- Bus scheduling (best turn 5): reward 218,246.00; avg passenger waiting time 179.09 s; fuel 102,208.54 g; on-time rate 0.10; 1 module improved vs. baseline (bus).

Your Reflection Task (Multi-turn, using DATA_ANALYSIS):

1. Enter a short reflection phase (up to **5** DATA_ANALYSIS turns).
2. Re-use cached sandbox data via `save_cache()/load_cache()/list_cache()`; do *not* run POLICY_PLANNING.
3. Final response: ACTION: REFLECTION_FINISH with **only** an updated memory list (JSON array of strings in a json block).

Final Output Format (Last Reflection Response with REFLECTION_FINISH):

ACTION: REFLECTION_FINISH

```
[
  "INSIGHT 1",
  "INSIGHT 2",
  "...",
  "INSIGHT N"
]
```

Important:

- Return a valid JSON array (max. 10 items): keep/refine prior memory, add new insights, drop unwanted items by omission.
- One sentence per item; robust, transferable strategies; no extra commentary outside the JSON array.

Example reflection outputs

Agent Response: Procedural Spatiotemporal Memory Summary

ACTION: DATA_ANALYSIS

```
# Reflection DATA_ANALYSIS (abbreviated): post-session pattern mining
print("=== REFLECTION: NETWORK PATTERNS ===")
try:
    cached_data = load_cache()
except Exception:
    cached_data = {}
intersection_types = {}
traffic_data = read_traffic_states(start_time=45001 - 3600, end_time=45001)
if 'error' not in traffic_data:
    snapshots = traffic_data.get('snapshots', [])
    time_periods = {}
    for snapshot in snapshots:
        hour = int(snapshot.get('time', 0) // 3600)
        bucket = time_periods.setdefault(hour, {"vehicles": 0, "waiting": 0, "n": 0})
        vehicles = snapshot.get('vehicles', [])
        bucket["vehicles"] += len(vehicles)
        bucket["waiting"] += sum(1 for v in vehicles if v.get('state') == 'waiting')
        bucket["n"] += 1
    # ... omitted: hourly printouts and intersection-type statistics
save_cache({
    "traffic_patterns": time_periods if 'time_periods' in locals() else {},
    "intersection_types": intersection_types if 'intersection_types' in locals() else {},
})
```

Agent Response: Procedural Spatiotemporal Memory Update

ACTION: REFLECTION_FINISH

```
[
  "Bus scheduling optimization during morning rush hour (07:30-08:00) should focus on aggressive headway reduction (25-30%) to reduce passenger waiting time, even if it increases fuel consumption.",
  "Signal timing optimization requires demand-based analysis rather than simple cycle time increases; focus on intersections with clear demand patterns (>10 queue length) for effective improvements.",
  "The network has short signal cycles (average 29.5 seconds) with most phases at minimum 10 seconds, indicating conservative signal timing by default.",
  "During rush hours, bus headways should be reduced to 60-180 seconds to minimize waiting time, with dwell time increases at high-demand stations.",
  "There is a trade-off between waiting time reduction and fuel consumption; prioritize waiting time reduction as the primary objective for public transit service quality.",
  "Bus scheduling optimization should target specific time ranges (e.g., 07:30-08:00) with time-based headway adjustments rather than uniform changes across all periods.",
  "Signal timing optimization should consider the optimization window's time period (morning rush) and adjust strategies accordingly, not just apply general increases.",
  "The network's bus routes have default headways of 600 seconds (10 minutes), indicating potential for significant improvement during peak demand periods.",
  "When optimizing multiple modules jointly, prioritize upstream modules (signal timing) first, but in this case, bus scheduling improvements had a more immediate impact.",
  "Future optimizations should use historical traffic data analysis to identify specific intersections and routes needing adjustment rather than blanket changes."
]
```

LLM-as-Judge Prompt: Policy Consistency and System-Level Traffic Efficiency Alignment

You are a professional traffic optimization expert. Please evaluate the following conversation between an LLM Agent and a traffic simulation environment. Score from 0-10 points.

Dimension 1: Multi-module Coordination Quality (0-5 points):

- This environment has the following control modules: {module_names_str}.
- Whether these modules are reasonably coordinated with each other.
- Whether the interactions and dependencies between modules are considered.
- Whether conflicts between modules are identified and avoided.

Dimension 2: Modeling Effectiveness (0-5 points):

- Whether the policy code correctly implements the optimization approach for {module_names_str}.
- Whether appropriate algorithms and parameters are used.
- Whether the data provided by the environment is fully utilized.
- Whether the agent iteratively improves the policy based on simulation feedback.

Please read the conversation carefully and provide your evaluation in the following STRICT format:

Score: [0-10 integer]

Brief Comment: [Your brief comment in 1-2 sentences]

Example output:

Score: 7

Brief Comment: The agent demonstrates good optimization strategy with reasonable parameter tuning, but could better utilize the simulation feedback for iterative improvement.

Conversation:

{conversation_text}

Your Evaluation: